Assignment 1 - Git & GitHub Fundamentals

Report

- **Full Name:** Kenza Dassaine
- **Student ID:** 168566
- **Course Code & Section:** CSC 332401
- **GitHub Username:** heretosee134

Repository: https://github.com/heretosee134/git-session1-heretosee134

1. Repository Setup

I unzipped the starter files into a project folder and ran git init, then git branch -M main to name the branch main. My .gitignore ignores local.env and all *.log files, with the line !important.log so that important.log stays visible to Git. git status confirms that local.env, debug.log and app.log are hidden while important.log is listed.

The GitHub repository is named git-session1-heretosee134 (my GitHub username); my local folder is still named git-session1-Kenzadssin.

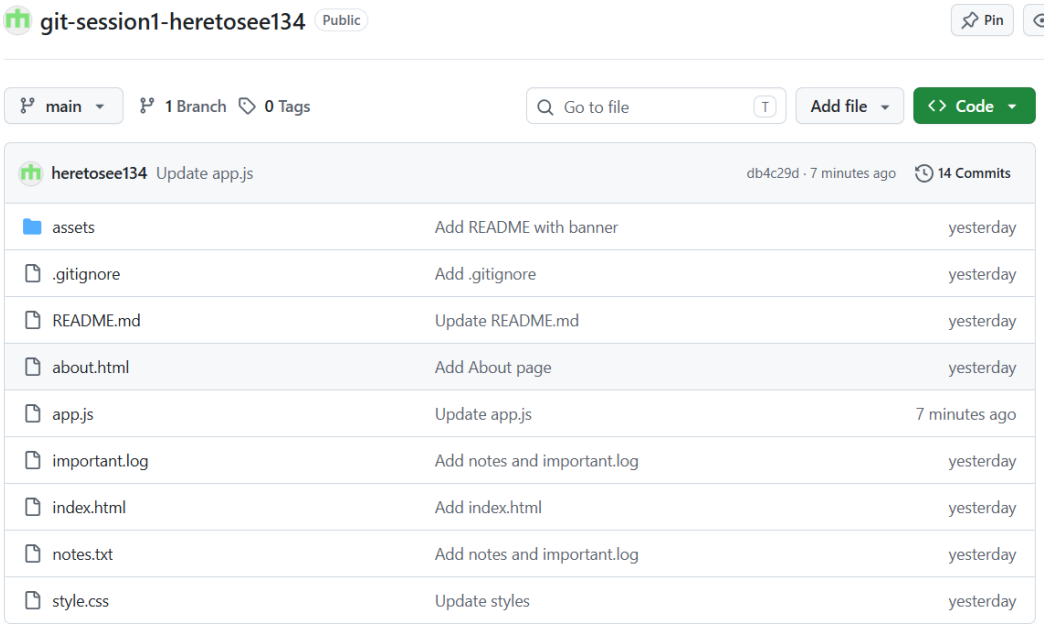
```
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin> git init
>> git branch -M main
Initialized empty Git repository in C:/Users/kenza/OneDrive/Desktop/Assignment1_Starter - group C/git-session1-Kenzadssin/.git/
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssingit status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        app.js
        assets/
        important.log
        index.html
        notes.txt
        style.css

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin>
```

git status after creating .gitignore: local.env, debug.log and app.log are ignored, important.log is visible



Final repository on GitHub: all project files present, no local.env or ignored .log files

2. README & Commit History

My README.md has my name, GitHub username, a short description, the technologies, one code block, one link (Pro Git Book) and the provided banner image. I made small, separate commits for each file: .gitignore, index.html, styles, app.js, README with banner, and notes with important.log.

```
 create mode 100644 README.md
 create mode 100644 assets/project-banner.svg
warning: in the working copy of 'important.log', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'notes.txt', LF will be replaced by CRLF the next time Git touches it
[main b9e55ff] Add notes and important.log
 2 files changed, 3 insertions(+)
 create mode 100644 important.log
 create mode 100644 notes.txt
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin> git log --oneline
b9e55ff (HEAD -> main) Add notes and important.log
54a46e4 Add README with banner
5ffd2cb Add app.js
52660c8 Add styles
83a7bac Add index.html
2420740 Add .gitignore
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin>
```

git log --oneline showing the first 6 commits

README

Git Session 1

Name: Kenza

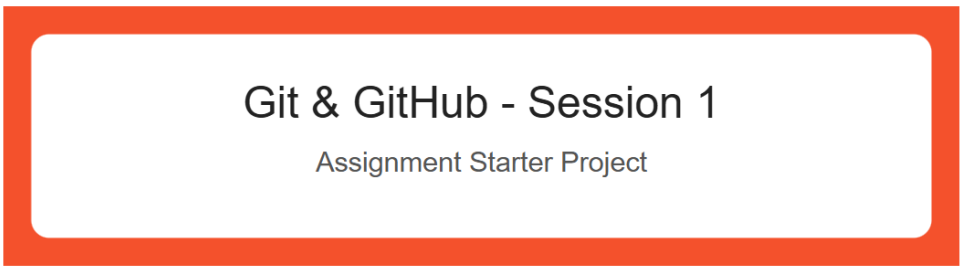
GitHub Username: heretosee134

A small web project used to practice Git and GitHub.

Technologies: HTML, CSS, JavaScript, Git

```
git status
```

Pro Git Book



Rendered README on GitHub, including the banner image

3. Selective Staging

I edited style.css and app.js at the same time, so git status showed both as modified. I then ran git add style.css and committed only that file ("Update styles"). The second git status shows that app.js is still modified and unstaged.

```
52660c8 Add styles
83a7bac Add index.html
2420740 Add .gitignore
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-s
ession1-Kenzadssin> git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory
)
        modified:   app.js
        modified:   style.css

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-s
ession1-Kenzadssin>
```

Both style.css and app.js modified

```
no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin> git add style.css
>> git commit -m "Update styles"
>> git status
warning: in the working copy of 'style.css', LF will be replaced by CRLF the next time Git touches it
[main 6763b61] Update styles
 1 file changed, 1 insertion(+)
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.js

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin>
```

After committing only style.css: app.js is still modified and not staged

4. Branch & Merge

I temporarily stashed my unstaged app.js change, then created the about-feature branch with git switch -c about-feature. On that branch I added about.html and committed it ("Add About page"). I switched back to main and merged with git merge --no-ff, which creates a visible merge commit, then restored app.js with git stash pop. The graph shows the branch and the merge commit.

```
* e53fe19 (HEAD -> main, origin/main, origin/HEAD) Update README.md
* 6a29524 Fix README formatting
* 17a703f Fix GitHub username in README
* 17be05f Add README content
*   63a04e4 Merge about-feature into main
|\
| * 41ac889 (about-feature) Add About page
|/
* 6763b61 Update styles
* b9e55ff Add notes and important.log
* 54a46e4 Add README with banner
* 5ffd2cb Add app.js
* 52660c8 Add styles
* 83a7bac Add index.html
* 2420740 Add .gitignore
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin>
```

git log --oneline --graph --all: about-feature merged into main

5. Remote Changes

A commit saves a snapshot of my changes in my local repository only, while a push uploads my local commits to the remote repository on GitHub. A fetch downloads new commits from GitHub without changing my working files, while a pull is a fetch followed by merging those changes into my current branch.

I edited README.md directly on GitHub, then ran git fetch. git status reported that local main was behind origin/main by 1 commit, and git log main..origin/main showed the missing commit. My local files had not changed yet. After git pull (a fast-forward), the README was updated locally.

```
no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin> git fetch
>> git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.js

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin> git log --oneline main..origin/main
e53fe19 (origin/main, origin/HEAD) Update README.md
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin>
```

After git fetch: local main is behind origin/main by 1 commit

```
  (use "git pull" to update your local branch)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.js

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin> git log --oneline main..origin/main
e53fe19 (origin/main, origin/HEAD) Update README.md
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin> git pull
Updating 6a29524..e53fe19
Fast-forward
 README.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin>
```

git pull fast-forwards main to the GitHub commit

Finally, I showed an unstaged diff (git diff) and a staged diff (git diff --staged) of my change to app.js, then committed and pushed it.

```
Updating 6a29524..e53fe19
Fast-forward
 README.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin> git diff
diff --git a/app.js b/app.js
index 603d28a..0ae1da6 100644
--- a/app.js
+++ b/app.js
@@ -3,3 +3,4 @@ const statusMessage = document.getElementById("status-message");
 if (statusMessage) {
   statusMessage.textContent = "Starter project is ready.";
 }
+// updated script
\ No newline at end of file
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin>
```

Unstaged diff of app.js (git diff)

```
 }
+// updated script
\ No newline at end of file
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin> git add app.js
>> git diff --staged
diff --git a/app.js b/app.js
index 603d28a..0ae1da6 100644
--- a/app.js
+++ b/app.js
@@ -3,3 +3,4 @@ const statusMessage = document.getElementById("status-message");
 if (statusMessage) {
   statusMessage.textContent = "Starter project is ready.";
 }
+// updated script
\ No newline at end of file
PS C:\Users\kenza\OneDrive\Desktop\Assignment1_Starter - group C\git-session1-Kenzadssin>
```

Staged diff of app.js (git diff --staged)

6. Fair Use of AI

I used Claude (an AI assistant) to get step-by-step guidance for this lab. I asked how to start the assignment, how to use VS Code and its terminal, how to do selective staging, create and merge a branch,

connect the local repository to GitHub, and how to read errors. I accepted the general sequence of commands, but I had to correct some things along the way: my first push failed with "Repository not found" because my GitHub username was different from the one I had assumed, so I fixed the remote URL and renamed the repository. I also found that my first README commit was empty because I had not saved the file, and that the banner image line in the README was missing the "!" so it showed as a link instead of an image. I verified everything by running each command myself, reading the output of git status, git log and git diff, and checking the final result on GitHub.

7. Sources & Reflection

Sources: the assignment instructions and starter files provided for Session 1, and the Pro Git book (https://git-scm.com/book).

The hardest concept for me was the difference between the working directory, the staging area and a commit, and between fetch and pull. Staging lets me choose exactly what goes into each commit, and fetch lets me see what is on GitHub before it changes my files.